

RL Learning • 强化学习

Part 1 — 从建模到求解 / From Modeling to Solving

诊断方法论 · RLHF → PPO 完整推导 · General RL 对比 | 重点在 cost function 之后「如何一步步求解」

目录

- 1 RL 诊断方法论:模型跑歪时,问题在哪一层
- 2 一道诊断题:数据中心冷却(reward misspec 的活标本)
- 3 同一个病,任何 ML 都有 —— 为什么 RL 更严重
- 4 RLHF → PPO:从建模到 cost function
- 5 最优 policy:泛函求得玻尔兹曼分布
- 6 核心:policy gradient 怎么求解(cost 之后的部分)
- 7 从裸梯度到 PPO:三步补丁
- 8 一个完整数值例子:走一遍 PPO 更新
- 9 串行 vs 并行 · 生成机制 · value 定义 · General RL 对比 · 网络梳理

1 RL 诊断方法论:模型跑歪时,问题在哪一层

用 RL(或任何 ML)解决实际问题时,一个反复出现的核心问题是:模型行为很奇怪,到底哪里出了错? 这不是背公式的问题,而是**诊断能力**——能不能快速定位病灶在哪一层。

核心诊断轴

一个 learning 系统可以拆成四层:目标 (reward/loss) × 容量 (model class) × 数据/环境 × 优化。跑歪了,就是这四层之一(或耦合)出了问题。关键技能是**快速定位在哪一层**,因为四层的修法完全不同,而且经常被误诊。

第一类:目标本身错了(reward misspecification)—— 最常见、最隐蔽

症状:模型 loss/reward 涨得很好,但你想要的行为没出现,甚至出现荒谬行为。

判据:如果把模型学到的策略当成对目标的「正确解」,它其实真的最大化了你写下的目标——只是你写下的目标 ≠ 你心里想要的。**模型没错,你的目标函数错了。**

- **CoastRunners 赛艇**(OpenAI):reward = 得分,本以为得分 $\propto$ 完成比赛。结果 agent 原地绕圈刷补给分,永远不冲线,得分反而更高。模型完美优化了 reward,是 reward 编码错了意图。
- **抓取任务**:reward = 机械臂位置在物体和相机连线上。agent 学会把手放在相机和物体之间**假装**抓到了。
- **RLHF 里的 sycophancy**:reward model 打分高 = 讨好、啰嗦、自信。policy 学会迎合 reward model 的偏好而非真答对。

修法:不是加容量、不是调 lr —— 是**改 reward**。误诊成「模型不够强」就白烧钱。

第二类:需要外部激励(reward 太稀疏 / 探索不到)

症状:模型什么都学不到,卡在零 reward。不是学歪,是根本没起步。

判据:目标是对的,但 reward 信号太远、太稀,随机探索碰不到第一个正反馈,梯度长期为零。经典案例:Montezuma's Revenge——纯 DQN 几乎零分,解法是加 curiosity / intrinsic motivation,给「探索到新状态」发奖,把稀疏问题变稠密。

关键判别:第一类学到了东西但学歪;这类根本没学到东西。症状不同:歪 vs 空。

第三类:容量不够(model class 装不下真解)

判据:不是目标错、不是探索不到,是假设空间里根本没有那个正确函数。例:线性模型学 XOR;太浅的 shallow tower 装不下 position 和 x 的交互。别急着往这诊断——90% 被归咎为「容量不够」的其实是目标或数据问题。最想看到的是你不要第一反应就加参数。

第四类:学出 hack(specification gaming)

第一类的放大版:利用仿真器 bug(碰撞检测数值 bug 把自己弹射出地图)、利用评测漏洞(让评测崩溃,崩溃判 0 惩罚 < 正常失败)、钻分布缝隙。本质:optimizer 是没有常识、极度诚实的钻空子机器——你留任何缝它都会钻。根源都是 spec 和 intent 之间的 gap。

现场诊断话术(拉分点)

遇到「模型行为很奇怪」,不要直接跳修法,按顺序排查、说出声:

1. 它是学歪了还是没学到?(misspecification vs 稀疏探索)
2. 如果学歪了,它是不是其实完美优化了我写的目标?(→ reward 错,不是模型错)
3. 训练误差降下去了吗?(没降 → 容量/优化;降了但行为怪 → 目标/数据)
4. 这个 hack 在训练分布里 reward 真的更高吗?(是 → spec gaming,改 spec)
5. 它利用的是真实信号还是环境/评测的 bug?(→ 修环境还是修目标)

核心洞察:优化器永远是对的,它精确地给你要的东西。模型「学歪」几乎从来不是模型的错,是目标、环境或数据里留了缝。第一反应该是审计 spec,不是加容量。

2 一道诊断题:数据中心冷却(reward misspec 的活标本)

场景:RL agent 控制数据中心冷却。action = 调各机组制冷功率, reward = -(总能耗),目标省电。硬约束:任何机柜温度超过 $T_{\max}$ 给大负 reward - M。

症状:训练曲线漂亮,比人工基线省 18% 电。但压力测试发现:agent 大部分时段把制冷压到近零,只在温度即将触顶前一刻猛开、贴着 $T_{\max}$ 反复横跳。能耗低,但机组频繁急启急停,缩短硬件寿命。

定位:这是 目标(reward)漏了,不是模型学歪。agent 精确地、诚实地最大化了 -能耗,贴顶横跳真的能耗最低,它优化对了。错的是 reward——只写了能耗,没写「硬件寿命/安全裕度」这两个真正在乎的成本。

这个区分是方向性的:说「模型歪了」会让你去修模型(加容量/换结构);说「目标漏了」才会让你去修 reward。诊断错方向,修法全错。

病根 vs 表象:「横跳」只是**表象**,病根是「**贴着 $T_{\max}$ 运行**」。横跳是贴顶的副作用。

要罚哪个量:工程师担心的是压缩机的急启急停,那是 **action(功率指令)** 在抖,不是温度在抖。该罚 action smoothness $\|a_t - a_{t-1}\|$,直接命中磨损硬件的量。(注:功率与温度强耦合的健康设备上,罚温度变化率也能间接传导,但罚 action 更直接、更少依赖「耦合健康」前提。)

只加 smoothness 够不够:不够。加了 action-smoothness 后,agent 的新策略几乎必然是**平滑地、稳稳地常年贴着 $T_{\max}$ 跑**:不横跳了(躲过 smoothness 罚),能耗还是最低(还在贴顶省电),但机组常年顶格、零安全裕度,任何扰动一冲就超温。**治了横跳,没治贴顶,还治出个更隐蔽的版本。**

完整答案分三层:(1) action smoothness 治急启急停(表象);(2) soft barrier / 带 buffer 的软上界治「贴顶」(病根,改 reward 形状:悬崖→斜坡);(3) 对每个新加的项再问一遍新缝—— λ 太大 agent 会过度保守、制冷常开、省电目标废掉。**每个 correction 都可能催生新 hack,这才是分水岭。**

治本:病根是**悬崖式 reward**($-M$ 硬阈值)制造了「只要不超线,越贴越赚」的激励。真正的修法是**把悬崖换成斜坡**:带 buffer 的软上界(soft barrier),温度越接近 $T_{\max}$ 越早、越平滑地递增惩罚,从结构上消除「贴临界最优」这个激励。

3 同一个病,任何 ML 都有 —— 为什么 RL 更严重

这个病**不是 RL 专属**。所有 ML 都在干一件事:optimizer 精确最大化你写下的目标。只要「写下的」和「想要的」有缝,optimizer 就去钻。

- **监督学习:**shortcut / spurious correlation。「狼 vs 哈士奇」分类器其实在看**背景有没有雪**;判肺炎的 CNN 学会看片子来自哪台机器。
- **LLM:**reward hacking / sycophancy,或刷 benchmark。
- **RL:**赛艇绕圈刷分、钻仿真器 bug。

三个长得不一样,**底层同一句话:目标留了缝,优化器钻了。**

为什么 RL 更严重:四个放大器

1. **闭环 vs 开环。** 监督学习数据固定,缝有限。RL 的 agent 自己生成数据,能主动把自己推进到能钻空子的角落。**能改变自己数据分布的系统,能把缝越钻越大。**
2. **时间累积 vs 单步。** 监督学习一个错预测损失一个样本,独立。RL 是序列,早期偏差复合放大(compounding error)。
3. **reward 比 label 稀疏、间接。** label 逐点钉死正确答案,缝小。reward 远期、标量、聚合,同一个 reward 能被无数种行为达到。**目标越粗,缝越大。**
4. **探索主动搜索 hack。** 监督学习被动拟合;RL 的 exploration 主动到处试,工作就是找高 reward 路径——包括钻空子的路径。

一句话:病是同一个(优化器诚实地钻目标的缝)。监督学习叫 shortcut learning,LLM 叫 reward hacking,RL 叫 specification gaming。RL 更严重是因为同时占了四个放大器。**监督学习的缝是静态有限的,RL 的缝是能被 agent 主动挖大的。**

4 RLHF → PPO:从建模到 cost function

大多数教程讲 PPO 到 cost function 就停了,没讲后面怎么一步步求解。这一节先建 cost,第 6 节起专讲求解。

第一步:唯一能拿到的信号是「相对偏好」

人没法给每个回答标绝对分数(一句话值几分说不清),但能给两个回答挑哪个更好。相对比较比绝对打分可靠得多。所以手上真正的原料是 (x, y^+, y^-) :问题、更好的回答、更差的回答。

第二步:把「人判偏好」写成概率

「人觉得 y^+ 更好」是个概率,取决于两回答「好的程度」差多少。给每个回答一个分 $s(y)$,用 sigmoid 把分差映射到 $[0, 1]$:

$$P(y^+ > y^-) = \sigma(s(y^+) - s(y^-))$$

这就是 **Bradley-Terry 偏好模型**。只要想把「人判哪个好」写成可优化的式子,就必须给每个回答一个「好的程度」的数——那个数就是 reward,换个名字而已,绕不开。

第三步:打分器 = pairwise 的 learning-to-rank

$s(y)$ 怎么训?正是 ranking 的 pairwise 范式(RankNet):**point-wise** 给分(一段话进网络,输出标量 $s(y)$)、**pairwise** 求梯度:

$$\text{loss}_{\text{RM}} = -\log \sigma(s(y^+) - s(y^-))$$

RLHF 的 reward model 训练 = **Bradley-Terry = RankNet = pairwise LTR**,三个名字一个东西。前向 point-wise,反向 pairwise(梯度经 $s(y^+) - s(y^-)$ 流回,此消彼长)。训完 s 冻住当裁判; y^+ / y^- 到此退场,后面 policy 训练不再碰它们。

第四步:policy 的目标 + KL 约束

policy π_θ 生成的回答,被 s 打分越高越好: $\max_\theta E_{y \sim \pi_\theta} [s(x, y)]$ 。但只 **max** 分数会出事: s 不完美,policy 会钻它误判高分的角落、把语言能力学坏。加约束别离基座 π_{ref} 太远:

$$\max_\theta E_{y \sim \pi_\theta} [s(x, y) - \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}]$$

两块:**刷分**(s)减 **偏离基座的惩罚**。到这里还没有 PPO、没有 clip。这只是一定要优化的目标本身。

两个「别跑太远」别混

	锚谁	干什么
KL 项 (β)	最初的 π_{ref} (SFT 基座)	防语言能力崩、防钻 RM 漏洞
clip (ϵ)	上一轮的 π_{old}	防单步更新过大、训练崩

KL 锚训练全程的起点(SFT,不变);clip 锚上一步(π_{old} ,每次采样刷新)。不同对象、不同目的。

5 最优 policy:泛函求导得玻尔兹曼分布

在讲「怎么数值优化」前,先看目标的闭式最优解——它会告诉我们为什么必须退回采样。对固定 x ,最大化 $E_{y \sim \pi}[s] - \beta KL(\pi \parallel \pi_{ref})$,带归一化约束,上拉格朗日,对 $\pi(y)$ 求泛函导数令 $= 0$,解出:

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{ref}(y | x) \exp\left(\frac{1}{\beta} s(x, y)\right), \quad Z(x) = \sum_v \pi_{ref}(y | x) e^{s(x, y)/\beta}$$

这是 **玻尔兹曼 / Gibbs 分布**——以基座 π_{ref} 为底,被打分 s 指数加权,温度 β 。和物理的最大熵模型同构。

为什么不能直接用这个闭式解: $Z(x) = \sum_y \pi_{ref}(y) e^{s/\beta}$ 要对所有可能的输出 y 求和——对 LLM 而言 y 是所有可能的句子,数量是词表大小的指数级。 **$Z(x)$ 算不出来**。所以 π^* 写得解析式、却算不出数值,没法当 target 去 backprop。

绕过 Z 的三条路:(1) MCMC 采样(不需 Z ,但参数没动、要用还得重跑);(2) policy gradient + 采样估梯度(PPO 走这条);(3) 数学上消掉 Z (pairwise 相减时 Z 抵消 $\rightarrow$ DPO,回到 $y^+ - y^-$,不做 RL)。

6 核心:policy gradient 怎么求解(cost 之后的部分)

本文重点。目标: $J(\theta) = E_{y \sim \pi_\theta}[R(x, y)]$, 其中 $R = s(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$ 。

先厘清 $\pi_\theta(y)$ 到底是什么

$\pi_\theta(y | x)$ 不是「把 y 输入得到一个分数」,而是生成整串 y 的概率:

$$\pi_\theta(y | x) = \prod_{t=1}^L \pi_\theta(y_t | x, y_{<t})$$

这是链式法则(每词条件于前文),不是独立假设。连乘的形式像独立,但每个因子带 $|y_{<t}$ 就不是。 θ 是网络参数(不是隐变量),明确、可导地定义 y 上的分布。单个 y 的概率 $\pi_\theta(y)$ 有闭式、可算可导(连乘 softmax 里选中词的概率);没有闭式的是全局归一化 Z 和采样操作。policy gradient 只用可算那部分。

求梯度:log-derivative trick

写成求和,乘积法则: $\nabla_\theta J = \sum_y \nabla_\theta \pi_\theta(y) R(y) + \sum_y \pi_\theta(y) \nabla_\theta R(y)$ 。

第 ① 项出现卡点: $\nabla_\theta \pi_\theta(y)$ 不是期望形式,没法采样。用 **log-derivative trick** $\nabla_\theta \pi_\theta(y) = \pi_\theta(y) \nabla_\theta \log \pi_\theta(y)$:

$$\textcircled{1} = E_{y \sim \pi_\theta}[R(y) \nabla_\theta \log \pi_\theta(y)]$$

第②项: $\nabla_{\theta} R = -\beta \nabla_{\theta} \log \pi_{\theta}$, 而 $E_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}] = \nabla_{\theta} \sum_y \pi_{\theta} = \nabla_{\theta} 1 = 0$, 期望为零。

$$\nabla_{\theta} J = E_{y \sim \pi_{\theta}}[R(x, y) \nabla_{\theta} \log \pi_{\theta}(y)] \approx \frac{1}{N} \sum_{i=1}^N R(x, y^{(i)}) \nabla_{\theta} \log \pi_{\theta}(y^{(i)}), \quad y^{(i)} \sim \pi_{\theta}$$

关键澄清: y 是常量, 不对它求导

在 $J = \sum_y \pi_{\theta}(y) R(y)$ 里, y 是**求和遍历的哑标**(跑遍所有句子), 不是 $y(\theta)$ 这个函数。对每个固定 y , $R(y)$ 就是个**确定的数**, 不含 θ , 所以 $\nabla_{\theta} R(y) = 0$ 精确成立, 不是近似。

θ 「影响采样出哪个 y 」精确地等于「 θ 影响每个 y 的概率 $\pi_{\theta}(y)$ 」, 这份影响**全部**被第①项抓走。

$\nabla_{\theta} y$ 从未出现, 不是被弄没的——是它本就不该被写出来。

关于「离散不可导」的澄清: 离散采样的梯度能算、有多种无偏估计器(score function/REINFORCE、Gumbel-Softmax、measure-valued 等)。RL 用 score function 估计器——对离散合法, 只需 $\nabla_{\theta} \log \pi_{\theta}(y)$ 可导(对数概率永远可导), 绕开 $\nabla_{\theta} y$ 。所以不是「离散不能求导」, 是 policy gradient 选择不穿过采样、把 reward 当外部常数, 只对概率 π_{θ} 求导。

直觉: reward-weighted maximum likelihood

$\nabla_{\theta} \log \pi_{\theta}(y)$ 是「让 model 更可能生成 y 」的**参数方向**(向量, 不是标量)。乘上 advantage A : $A > 0$ (好回答) 正向加权、**推高**其概率; $A < 0$ (差回答) 负向、**压低**。所以 policy gradient 就是**按 reward 加权的最大似然**。

7 从裸梯度到 PPO: 三步补丁

补丁一: 减 baseline(降方差)

裸梯度里 R 是绝对分, 若都在 100 上下, 乘 $\nabla_{\theta} \log \pi_{\theta}$ 后信号淹没在噪声里。从 R 减一个和 y 无关的基准 $b(x)$, 梯度期望不变 ($E[b \cdot \nabla_{\theta} \log \pi_{\theta}] = b \cdot 0 = 0$)。取 $b = V(x)$ 得 **advantage** $A(x, y) = R(x, y) - V(x)$ 。

一个精确点(常被讲错): 减常数 baseline, R 本身的方差不变(常数平移不改方差)。真正降的是 $R \cdot \nabla_{\theta} \log \pi_{\theta}$ 这个乘积的方差——因为系数绝对大小从 ~ 100 缩到 ~ 0 , 它对 $\nabla_{\theta} \log \pi_{\theta}$ 波动的**放大倍数**变小了。最优 baseline 其实是 $\frac{E[(\nabla \log \pi)^2 R]}{E[(\nabla \log \pi)^2]}$, $V(x)$ 是好用近似。降方差是**通过与 $\nabla \log \pi$ 的相互作用**, 不是单纯缩小 R 。

$V(x)$ 谁给? 再训个网络(critic) $\rightarrow$ actor + critic = **actor-critic**。或(GRPO)同一 x 采一组回答, 用**组内**平均分当 $V(x)$, 不训 critic。

补丁二: importance sampling(复用旧样本)

采样最贵, 采一批只更新一步太浪费。想让一批样本**多更新几步**, 但每更新一步 θ 就变、样本就「过期」(来自旧分布 π_{old})。用概率比 $\rho = \pi_{\theta}(y) / \pi_{\text{old}}(y)$ 修正: $J \approx E_{y \sim \pi_{\text{old}}}[\rho A]$ 。

importance sampling 是精确恒等式,不是近似:

$$E_{y \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(y)}{\pi_{\text{old}}(y)} f(y) \right] = E_{y \sim \pi_{\theta}} [f(y)]$$

ρ 一乘,期望下面从 π_{old} 精确换成 π_{θ} 。拿旧样本 + ρ 估新梯度,期望上无偏。

ρ 修什么:它不是把旧梯度混进新梯度(不是动量/平滑),而是拿旧样本算全新的梯度, ρ 修「样本来自旧分布」这个偏差。好比汇率换算:旧样本用「旧货币」(π_{old})计价, ρ 换算成「新货币」(π_{θ})计价,算出的是纯粹新梯度,旧梯度一点没进去。

不用 ρ 会怎样:第二次更新时 θ 已变,若不乘 ρ ,算的是 $E_{y \sim \pi_{\text{old}}} [A \nabla \log \pi_{\theta_1}]$,期望下面是错的分佈(应是 π_{θ_1}) → 有偏梯度。表现为:对已推高的好样本过度推高、对已压低的差样本过度压低,几次后过拟合、可能崩。

补丁三:clip(限幅防爆)

ρ 有个毛病: π_{θ} 走太远, ρ 极大/极小,IS 方差爆、单样本主导梯度、训练崩。把 ρ 夹在 $[1 - \epsilon, 1 + \epsilon]$:

$$L^{\text{PPO}} = E[\min(\rho A, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) A)]$$

clip 夹的是 ρ (概率比),不是 A (advantage)。这个区分是核心: $A = R - V$ 是信号本身(这回答比平均好多少),不该限制; ρ 是参数更新的步子,限制它是为了不让单个样本贡献失控(ρ 巨大时一个样本绑架整个梯度)。所以 clip 保证任何单个样本对梯度的贡献不过大,通过夹 ρ 、留 A 实现。

三件套总览

零件	干什么	跟着什么变
advantage A	定方向(好推差压)、定力度	信号本身,不动
$\rho = \pi_{\theta} / \pi_{\text{old}}$	把权重从旧分布纠到新分布(无偏)	每轮在新 θ 处重算
clip	限幅,防单样本贡献失控	夹 ρ 在 $[1 - \epsilon, 1 + \epsilon]$

$\nabla_{\theta} \log \pi_{\theta}$ 永远跟着最新 θ 变; ρ 只修每个样本的权重,不碰那个导数方向。

8 一个完整数值例子:走一遍 PPO 更新

采样阶段(用 π_{old} ,贵,只做一次)

设 $M = 2$ 个 prompt,每个采 $N = 4$ 个回答,共 8 样本。打分: $x_1 : R = [2, 5, 1, 4], x_2 : R = [3, 3, 6, 0]$ 。同时记下每样本在 π_{old} 下的概率。

算 advantage(组内平均当 baseline)

$\bar{R}_1 = 3 \Rightarrow A = [-1, +2, -2, +1]; \bar{R}_2 = 3 \Rightarrow A = [0, 0, +3, -3]$ 。好于同题平均 $A > 0$ (推高), 差的 $A < 0$ (压低)。

更新阶段

第 1 次($\theta = \theta_{old}$, 每个 $\rho = 1$): $g_1 = \frac{1}{8} \sum_i A_i \nabla_{\theta} \log \pi(y_i)|_{\theta_{old}}$ 。不触发 clip。走一步 $\rightarrow \theta_1$ (y_2 被推高、 y_8 被压低)。

第 2 次($\theta = \theta_1$, 还是这 8 样本): 重新前向算新概率, $\rho_i = \pi_{\theta_1}(y_i) / \pi_{old}(y_i)$ 不再是 1。例: y_2 (被推高) $\rightarrow \rho_2 = 1.3$; y_8 (被压低) $\rightarrow \rho_8 = 0.6$ 。 $\rho_2 = 1.3 > 1.2 \Rightarrow$ clip 触发夹到 1.2——含义: y_2 推够了, 别再使劲。求梯度、走一步 $\rightarrow \theta_2$ 。3、4 次同理。榨干后回采样阶段重采。

「一批样本更新几次」是超参(epochs per rollout, 常见 2-4), 人为定。更新越多 $\rightarrow \theta$ 离 π_{old} 越远 $\rightarrow$ 越多样本 ρ 出界被 clip $\rightarrow$ 收益递减。注意: 不用 $\rho \neq 1$ 「当 $\rho = 1$ 硬更新」(后者是有偏做法), 而是「只能每步重采、更新一次」。

怎么算 $\pi_{\theta_1}(y)$: y 不变, 重新前向查概率

y 固定(如 [猫, 很, 可爱]), 用新参数 θ_1 前向, 每步 softmax 查该词概率连乘: $\pi_{\theta_1}(y) = 0.5 \times 0.4 \times 0.6 = 0.12$ 。旧 θ_{old} 下连乘 = 0.10, 故 $\rho = 0.12 / 0.10 = 1.2$ 。

为什么这步便宜: 算 $\pi_{\theta_1}(y)$ 是一次并行前向(整个 y 已知, 所有位置一次喂入并行算, teacher forcing)。而采样是自回归、逐 token 串行, 慢得多。所以宁可用 ρ 重算旧样本的新概率, 也不重新采样。

9 串行/并行 · 生成机制 · value · General RL · 网络梳理

串行 vs 并行: 同一个 model, 两种时机

为什么生成必须串行、算概率却能并行? 关键一句: 下一步的输入, 是不是要等上一步的输出。

	手上有什么	下一步输入哪来	所以
采样(生成)	只有 x , 词要现生	依赖上一步刚抽出的词	必须等 $\rightarrow$ 串行
算概率(评估)	整句已知	直接从已知句子里取	不用等 $\rightarrow$ 并行

并行 $\neq$ 独立: 并行只是「因 y 已知, 所有位置同时算」; 每个位置内部通过 causal attention 仍只看前文($< t$)。条件依赖一点没少, 两种时机算出的概率完全一致。

生成机制: 给定 x 怎么一步步产出 y

不是「对 x 做一次 attention 就完」, 而是每生成一个词就把它加进序列、重新 attention: 序列 = $[x]$ $\rightarrow$ 做 causal self-attention $\rightarrow$ 取最后位置向量 $h_n \rightarrow h_n \times W_{unembed} \rightarrow$ logits $\rightarrow$ softmax $\rightarrow$ 抽词 = $y_1 \rightarrow$ 接回, 序列 = $[x, y_1] \rightarrow$ 在整个序列上重做 attention $\rightarrow$ 抽 $y_2 \rightarrow \dots$ 直到 EOS。

attention 作用的序列在增长:第 t 步作用在 $[x, y_1, \dots, y_{t-1}]$ 。x 永远在开头当锚点,每个生成的词都 attend 到 prompt 和自己前面的所有词——这是回答扣题、连贯的原因。(KV cache 缓存前面词的 K/V 不重算,是加速,不改逻辑。)

噪音、混沌与误差累积

自回归有混沌的种子(递归+非线性),但几个机制摠住它:prompt 当不动锚点、语言是强约束吸引子(语法语义自动纠错)、softmax 每步归一化防数值漂移、训练推向自恰低熵区。所以噪音有界,非指数飘飞;但摠不住时(high temperature、超训练长度、对抗 prompt、重复退化)真会飘飞。

训练稳、生成飘的不对称(exposure bias): 训练用 teacher forcing,每步喂真前文,错了立刻被纠正;生成时前文是自己产的,偏了没人拉。标准交叉熵不管这个累积——这正是 RLHF 存在的一个根本动机:交叉熵只管逐词、不管整段;RL 的序列级 reward 对整段生成打分,天然惩罚累积崩坏。

一层哲学澄清:attention 的结构(每词看全前文)对齐了语言概率的链式分解 $\prod_t P(y_t|y_{<t})$,这是它比 n-gram/RNN 强的根。但结构对齐 $\neq$ 就是真分布:它是可学习的逼近器,有误差。更底一层:「语言本身不是概率过程」这个前提本身存疑——概率建模是我们选择套在语言上的镜头,不是语言的本体属性。

Value function:定义的来由

$V^\pi(s) = E_\pi[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s]$ 。三个设计决定:未来累积(要长期好不是眼前好)、折扣 γ (无限和要收敛、远期不可靠、早拿更值钱)、期望(未来随机,取平均得可优化的确定标量)。

终极价值:满足 Bellman 递归 $V(s) = E[r + \gamma V(s')]$ 。有它不用真模拟到无限步,可迭代求解(DP、TD)。定义本身是数学本质(不是算力妥协);但「怎么估 value」有多种方法(MC/TD/n-step/GAE/DP/神经网络逼近),它们之间的取舍才是被算力、内存、数据、是否已知模型逼出来的。

General RL:对着 RLHF 看多步版

RLHF 是 general PPO 在「整段=一个动作、只有结尾一个 reward」下的单步塌缩版。三件套两边完全相同,只需把每个零件从「一个」展开成「逐时间步一串」,再补 GAE 估多步 advantage。

零件	RLHF(单步)	General RL(多步)
交互	一问一答	多步轨迹 $\tau = (s_0, a_0, r_0, \dots)$
动作	整段 y 当一个动作	每步一个 a_t
reward	只在结尾一个 $R(x, y)$	每步 r_t
目标 J	$E_y[R(x, y)]$	$E_\tau[\sum_t \gamma^t r_t]$
value	$V(x) =$ 回答平均(单步退化)	$V(s_t) =$ 多步折扣累积
advantage	$A = R - V(x)$ 一个	$A_t = Q(s_t, a_t) - V(s_t)$ 逐步
A 怎么估	直接减	GAE (沿未来累加)
ρ	整段一个	ρ_t 逐步
clip	clip 一个 ρ	clip 每个 ρ_t

General 目标与梯度: $J(\theta) = E_{\tau \sim \pi_\theta}[\sum_{t=0}^T \gamma^t r_t], \nabla_\theta J = E[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot A_t]$.

t 是未来,不是过去。 J 是对还没发生、按 π_θ 采样出来的轨迹求期望。训练时用「已跑完的轨迹」去估,那只是采样估计的手段——严格说估的是 $\nabla_\theta J$ 。未来靠 policy 实际跑/自我博弈采样估,不跑就没有。

GAE: $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (单步 TD 残差), $A_t^{GAE} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$ 。 λ 调 bias-variance: $\lambda = 0$ 全靠 value(偏、稳), $\lambda = 1$ 全靠实际回报(无偏、方差大)。RLHF 单步时这些全退化。

一句话: general PPO = 把 RLHF 的「单步」沿时间轴展开成「多步」。三件套完全相同;唯一实质新增的是 value 从「回答平均」还原成「多步折扣累积」、advantage 从「一减」变成「GAE」。啃透 RLHF 后,general 没有新的核心机制要学。

RLHF 里有几个网络(收尾梳理)

网络	是什么	训不训
policy π_θ	要训的 LLM 本身(最终产物)	训
reference π_{ref}	SFT 基座的冻结副本,只用来算 KL	不训
reward model s	打分器(pairwise 训好),当裁判	不训(冻结)
value / critic V_ϕ	估 baseline(GRPO 用组内平均代替)	训(非最终产物)

policy 网络就是那个 LLM。 policy 和 reference 是「同一个 LLM 的两个版本」——reference 是训练开始时 SFT 的冻结快照,policy 从它出发被 RL 不断更新。KL 项 $\log \frac{\pi_\theta}{\pi_{ref}}$ 量的是「训着训着的 LLM 离出发时的自己有多远」。其余网络都是辅助。

全文完 —— 从 reward 诊断到 PPO 求解。核心是:cost function 只是起点,
「Z 算不出 → 退回采样 → log-trick 出梯度 → baseline / ρ / clip 三步补丁」才是 PPO 的真正内容。